Case Prático: Diagnóstico e Solução de Problemas em uma API

Cenário

Você foi contratado para diagnosticar e solucionar problemas em uma API RESTful já existente. A API é responsável por gerenciar usuários e suas respectivas tarefas. Alguns usuários têm relatado falhas e lentidão ao utilizar os serviços.

Objetivo

Identificar e solucionar os problemas relatados, garantindo o correto funcionamento e bom desempenho da API.

Descrição do Problema

A API apresenta os seguintes sintomas:

1. Endpoint `GET /users` não possui nenhum tipo de filtro. Permitir filtrar pelo nome e id.
2. Senha dos usuários estão sendo retornados no endpoint GET /users, causando uma falha de segurança.
3. Não há um endpoint para recuperar dados de um único usuário.
4. Endpoint POST /users não está criptografando as senhas.
    1. Não há validação de parâmetros nesse endpoint. Podemos mandar qualquer informação no campo e-mail, por exemplo.
5. Lentidão no endpoint `GET /tasks?userId={id}` quando um usuário possui mais de 1.000 tarefas.
6. O endpoint `POST /users` permite criar usuários com emails duplicados.
7. O endpoint `DELETE /users/{id}` falha ao excluir usuários que possuem tarefas associadas.
8. O endpoint `DELETE /users/{id} está excluindo o registro fisicamente no BD. É possível implementar o soft delete?`
9. Logs importantes das operações (ex.: criação de tarefas, exclusão de usuários) não estão sendo gravados corretamente.
10. Não existe documentação dos endpoints, para que a equipe frontend possam consumir os serviços oferecidos pela API.

Tarefas

Você deve:

1. **Diagnosticar e corrigir os problemas listados**:
    - Explique a causa raiz de cada problema.
    - Proponha e implemente a solução para cada caso.

2. **Melhorar a API**:
    - Adicione validações robustas nos endpoints.

- Implemente uma camada de logs centralizada para registrar as operações.
- Garanta que ao excluir um usuário suas tarefas sejam tratadas corretamente (ex.: exclusão em cascata ou reatribuição a um administrador).

3. **Testar**:
    - Adicione testes para validar os endpoints corrigidos e os novos comportamentos implementados.

Requisitos Técnicos

- Você receberá:
    - O código da API em PHP com Laravel Lumen.
    - A estrutura do banco de dados (por exemplo, MySQL ou MongoDB).
    - Um arquivo `README.md` básico com instruções de execução.

- **Suas entregas devem incluir:**
    - Correções aplicadas no código.
    - Testes automatizados que comprovem que os problemas foram resolvidos.
    - Explicação técnica das causas e soluções aplicadas (em um arquivo SOLUTION.md).

Critérios de Avaliação

1. **Análise e Diagnóstico**:
    - Sua capacidade de identificar as causas dos problemas e propor soluções adequadas.

2. **Implementação de Soluções**:
    - Clareza e eficiência do código.
    - Resolução efetiva dos problemas.

3. **Boas Práticas**:
    - Organização do projeto.
    - Implementação de validações e tratamento de erros.
    - Documentação clara e objetiva.

4. **Performance**:
    - Melhorias no desempenho do endpoint `GET /tasks?userId={id}`.

5. **Testes**:
    - Cobertura e qualidade dos testes automatizados.

Diferenciais

- Implementar caching para otimizar consultas de tarefas.
- Refatorar a API para seguir princípios de arquitetura limpa.
- Opcionalmente, implementar autenticação e autorização básicas (ex.: JWT).

Entrega

- Faça o clone do repositório e envie as alterações em uma branch chamada feat/{first-name}.
- Inclua no repositório:
    - O código corrigido.
    - O arquivo SOLUTION.md detalhando suas análises e soluções.
    - Instruções no `README.md` para testar as alterações.